

Protocolo de Investigación Experimental

Aprendizaje de Rust: Meta-aprendizaje vs. Asistencia Inteligente Justo a Tiempo

Brahian Marín Giraldo
Biología 2026-2
Universidad CES

30 de julio de 2026

Resumen

El lenguaje de programación Rust impone un estricto modelo de propiedad, préstamo y tiempos de vida que previene errores de memoria, pero dificulta su aprendizaje. Este estudio empírico compara dos enfoques pedagógicos: el aprendizaje fundamentado en teoría (Aprender a Aprender) y el aprendizaje justo a tiempo asistido por IA (JIT). Con una muestra de 100 estudiantes de ingeniería, se miden métricas cuantitativas y cualitativas para evaluar retención, depuración y comprensión profunda. Los resultados esperados matizarán las afirmaciones de Raj Reddy sobre el rol de los tutores virtuales en la educación universitaria.

1. Introducción y Marco Teórico

Rust es un lenguaje de programación que garantiza la seguridad de la memoria mediante un sistema de *ownership*, *borrowing* y *lifetimes*. Aunque estas características lo hacen robusto, su curva de aprendizaje es pronunciada; muchos programadores luchan con el compilador `rustc`. La llegada de asistentes de IA como ChatGPT y Copilot ofrece una vía para superar estas barreras, proporcionando sugerencias y explicaciones bajo demanda.

La teoría de **Raj Reddy** sobre la *autodidaxia asistida por IA* sostiene que, en una era de abundancia tecnológica, los estudiantes pueden aprender de manera más eficiente si cuentan con tutores virtuales que los guíen en la resolución de problemas concretos. Reddy argumenta que la IA puede personalizar el aprendizaje y fomentar la curiosidad, pero advierte sobre el riesgo de una comprensión superficial si no se desarrollan habilidades metacognitivas.

Este estudio compara dos métodos pedagógicos para la enseñanza de Rust:

- **Grupo Control – Aprender a Aprender (AaA):** Recibe instrucción en teoría de tipos, semántica de lenguajes, modelos de memoria (heap/stack) y diseño conceptual, antes de codificar.

- **Grupo Experimental – Aprender Justo a Tiempo (JIT):** Se enfrenta directamente a la construcción de una aplicación, utilizando asistentes de IA para resolver errores del compilador conforme aparecen.

El objetivo es medir la efectividad de cada método en términos de retención, capacidad de depuración y comprensión profunda de los conceptos de memoria.

2. Metodología del Experimento

2.1. Muestreo y Participantes

- **Población:** Estudiantes de Ingeniería de Sistemas de universidades colombianas.
- **Muestra:** 100 estudiantes seleccionados mediante muestreo estratificado por nivel de conocimiento previo en programación (bajo, medio, alto). Se asignan aleatoriamente a dos grupos de 50.
- **Criterios de inclusión:** Haber cursado al menos un curso de programación estructurada; sin experiencia previa en Rust.
- **Duración:** 6 semanas, con sesiones de 4 horas semanales (24 horas totales).

2.2. Diseño Experimental

- **Variable independiente:** Método pedagógico (AaA vs. JIT).
- **Variables dependientes:**
 - Rendimiento en pruebas de código (corrección, eficiencia).
 - Tiempo de resolución de errores de borrowing.
 - Capacidad de refactorización sin ayuda.
 - Comprensión teórica (examen conceptual).
 - Número de invocaciones a la IA (solo grupo JIT).

2.3. Procedimiento

Semana 1–2 (Fase de instrucción):

- **Grupo AaA:** Clases magistrales sobre teoría de tipos, memoria (stack/heap), ownership, borrowing y lifetimes. Ejercicios guiados sin IA.
- **Grupo JIT:** Se les presenta un proyecto (ej. un analizador de texto) y se les anima a usar ChatGPT/Copilot para resolver los errores del compilador. No reciben teoría previa.

Semana 3–4 (Práctica supervisada): Ambos grupos trabajan en proyectos similares (pero no idénticos) con complejidad creciente. El grupo AaA no usa IA; el JIT puede usarla libremente. Se registran métricas (tiempo, invocaciones, errores).

Semana 5 (Evaluación formativa): Prueba práctica de depuración: se entrega un código con errores de ownership y lifetimes; deben corregirlo sin IA (ambos grupos). Se mide tiempo y precisión.

Semana 6 (Evaluación sumativa):

- Examen teórico (preguntas de opción múltiple y abiertas) sobre conceptos de memoria y borrowing.
- Proyecto final: construir una aplicación de consola en Rust sin asistencia (en laboratorio controlado). Se evalúa funcionalidad, estilo y manejo de memoria.

3. Plan de Recolección y Análisis Estadístico de Datos

3.1. Métricas Cuantitativas y Cualitativas

Cuadro 1: Métricas de evaluación

Métrica	Definición	Tipo
Tiempo promedio de corrección de errores de borrowing	Minutos empleados en resolver cada error de compilación relacionado con préstamos.	Cuantitativa
Tasa de éxito en proyecto final sin IA	Porcentaje de pruebas unitarias superadas en el proyecto final.	Cuantitativa
Número de invocaciones a la IA (solo JIT)	Conteo de consultas a asistentes durante las semanas 3–4.	Cuantitativa
Comprensión teórica de la memoria	Puntaje en examen conceptual (0–100).	Cuantitativa
Capacidad de refactorización	Evaluación cualitativa por pares ciegos de la claridad y eficiencia del código final.	Cualitativa (escala Likert)

3.2. Instrumentos

- **Registros automáticos:** Logs del compilador y de uso de IA (para el grupo JIT).
- **Cuestionarios:** Examen teórico diseñado por expertos en lenguajes.

- **Rúbrica de evaluación de código:** Criterios de legibilidad, uso correcto de ownership, manejo de errores.

3.3. Análisis Estadístico

- **Contraste de hipótesis:** Se utilizará la prueba t de Student para muestras independientes (si los datos son normales) o la U de Mann-Whitney (si no). Nivel de significancia $\alpha = 0,05$.
- **Hipótesis nula (H_0):** No hay diferencias significativas en el rendimiento entre ambos grupos.
- **Hipótesis alternativa (H_1):** El grupo AaA supera al JIT en al menos dos de las métricas principales (tiempo de corrección, examen teórico y proyecto final).
- **Análisis de correlación:** Para el grupo JIT, se explorará la relación entre el número de invocaciones a la IA y el rendimiento final, usando coeficiente de Pearson.
- **Análisis cualitativo:** Las evaluaciones de refactorización se analizarán mediante comparación de medianas (prueba de Wilcoxon).

4. Conclusiones Preliminares y Discusión Pedagógica

4.1. Resultados esperados

Si H_0 se rechaza, se confirmaría que la instrucción teórica previa (AaA) proporciona una base más sólida para enfrentar problemas complejos de memoria, mientras que el enfoque JIT podría generar una dependencia de la IA que limite la comprensión profunda. No obstante, es posible que el grupo JIT muestre mayor agilidad en tareas rutinarias, gracias al entrenamiento práctico intensivo.

4.2. Implicaciones para la teoría de Raj Reddy

Reddy sostiene que los tutores virtuales permiten un aprendizaje autodirigido y eficiente. Sin embargo, este estudio podría matizar su postura: la IA es una herramienta poderosa para resolver problemas concretos, pero no necesariamente fomenta la construcción de modelos mentales abstractos. Si el grupo AaA supera al JIT en comprensión teórica y refactorización, se sugeriría que el *aprender a aprender* debe preceder o acompañar el uso de asistentes, para evitar una ilusión de competencia".

Por otro lado, si el grupo JIT iguala o supera al AaA en el proyecto final, se reforzaría la idea de que el aprendizaje justo a tiempo, apoyado por IA, es viable y puede democratizar el acceso a lenguajes complejos. En cualquier caso, los resultados orientarán el diseño de futuros cursos de programación, equilibrando la instrucción teórica con el uso estratégico de herramientas inteligentes.

Referencias

- [1] Reddy, R. (2023). *Autodidaxia in the Age of AI: The Role of Virtual Tutors*. (Referencia hipotética; se recomienda citar la obra original).
- [2] The Rust Programming Language, *The Rust Book*. (2024).
- [3] S. Klusener et al., “Learning Rust: A Study of Novice Difficulties,” *ACM SIGCSE*, 2023.